

REPVIT REPRODUCTION

RepViT 轻量图像分类模型 复现、优化与多模型部署

2026 秋 one 团队 AI 算法组考核 · 答辩汇报

RepViT: Revisiting Mobile CNN From ViT Perspective · CVPR 2024 · arXiv:2307.09283

github.com/8ga-tech/RepViT-Reproduction



78.20%

ImageNet 子集 Top-1

92.34%

Pet-37 test Top-1

7.1e-06

重参数化 $\max|\Delta \text{logits}|$

100%

PyTorch↔ONNX Top-1 一致

六个考核环节全部交付，37 条验收项自检 0 失败

运行 → 迁移 → 优化 → 可视化 → 重参数化 → 部署，每个数字都可在仓库中溯源

01

官方模型评价

5 个型号 · 1000 张子集

M0.9 Top-1 78.20%

outputs/pretrained_eval/

02

M0.9 迁移训练

Pet 37 类 · 40 epoch

test Top-1 92.34%

checkpoints/baseline_best.pt

03

控制变量优化

4 方案 + 3 组消融

7 份配置 DIFF PASS

configs/opt_*.yaml

04

可视化与可解释性

五条曲线 · 混淆矩阵 · Grad-CAM

跨物种错误仅 3.56%

outputs/gradcam/

05

结构重参数化

BN 107 → 0 · Conv 126 → 103

max| Δ | 7.1e-06 · Top-1 32/32

outputs/reparam/

06

ONNX 多模型部署

6 个 ONNX · ORT CPU

P50 7.1 ~ 32.7 ms

onnx/*.onnx

M0.9 是 20 个 RepViT Block 的纯卷积流水线

形状由 forward hook 现测，非论文插图；分辨率 $224 \rightarrow 7$ ，通道 $3 \rightarrow 384$

输入

$224 \times 224 \times 3$ RGB

Stem

conv1 3×3 s2 $\rightarrow$ 24ch 112×112

conv2 3×3 s2 $\rightarrow$ 48ch 56×56

Stage 0

$2 \times$ RepViT Block 48ch 56×56

Stage 1

$2 \times$ RepViT Block 96ch 28×28

Stage 2

$14 \times$ RepViT Block 192ch 14×14

Block 数最多 —— 与论文「第三阶段布置更多 Block」一致

Stage 3

$2 \times$ RepViT Block 384ch 7×7

GAP $\rightarrow$ 384 维

Head RepViTClassifier (NormLinear) $\rightarrow$ 37 维

单个 RepViT Block

Token Mixer (空间混合)

$\text{RepVGGDW} = 3 \times 3 \text{ dw} + 1 \times 1 \text{ dw}$

Channel Mixer (通道混合)

1×1 升维 $\rightarrow$ GELU $\rightarrow 1 \times 1$ 降维

残差连接 + 部分 Block 带 SE

训练态 vs 推理态

训练态

3×3 dw 分支 + 1×1 dw 分支

两条分支各自带 BatchNorm

推理态

BN 折进卷积核后相加

$\rightarrow$ 单个 3×3 depthwise 卷积

名字里有 ViT，但结构里没有一个注意力算子

RepViT 借鉴的是轻量 ViT 的宏观架构选择，而非多头自注意力本身

标准 ViT	RepViT
空间混合Multi-Head Self-Attention	RepVGGDW (3×3 dw + 1×1 dw)
通道混合MLP / FFN	Channel Mixer (1×1 → GELU → 1×1)
下采样patchify (stride=16 大核)	Early Conv Stem (两组 stride=2)
归纳偏置弱，依赖大数据	强 (卷积自带局部性 / 平移等变)
ONNX 图内MatMul + Softmax	无注意力算子
全部算术节点	
Conv / Gemm / Add / Mul / Div / Clip / Relu	
实证: onnx/*.onnx 的 op_histogram 与 outputs/reparam/*_onnx_nodes.json	

四个关键设计

分离 Token / Channel Mixer
空间与通道容量可独立调节

扩张比降到约 2，同时加宽
省下的 MACs 换成通道数

SE 不做每块都放
全局池化的延迟不可忽略

简单分类头
单个 BN + Linear，省真实延迟

M0.9 实测: 20 个 Block
[2, 2, 14, 2]

官方权重在自建子集上的实际表现

1000 张自建分层子集（每类 1 张，种子 20260912） · 224×224 · batch 64 · FP32 · RTX 4060 Laptop

型号	参数量 (M)	MACs (G)	本机 Top-1	官方公布
RepViT-M0.9	5.489	0.847	78.20%	78.7%
RepViT-M1.0	7.303	1.143	79.90%	80.0%
RepViT-M1.1	8.803	1.358	79.90%	80.7%
RepViT-M1.5	14.644	2.308	82.80%	82.3%
RepViT-M2.3	23.689	4.574	83.10%	83.3%

预处理口径必须标

crop_pct = 0.875
Resize(256) + CenterCrop(224)

crop_pct = 0.95
Resize(235) + CenterCrop(224)

M0.9: 78.20 / 78.60
M1.0: 79.90 / 80.00

两套口径差 0.1~0.4 个点

必须声明的口径边界

本结果为「该自建验证子集上的实际运行结果」，不代表论文完整 ImageNet-1K 验证集结果。
子集为每类 1 张的分层抽样（1000 张），非考核方下发的指定子集；清单与哈希见 datasets/lists/imagenet_val_subset.txt
官方 iPhone 12 延迟 (0.9 / 1.0 ms) 与本机 CPU 结果不可横向比较

两套数据两套标签，三层划分零泄漏

37 类与 1000 类必须分别使用各自的标签文件；混用会让 Top-5 名称整体错乱

Oxford-IIIT Pet 划分（唯一划分脚本，seed=42）

trainval.txt 3680 行	pet_train.txt 2940 行 · 每类 73~80	pet_val.txt 740 行 · 每类恰好 20
test.txt 3669 行	pet_test.txt 3669 行 · 每类 88~100	test 只评价一次 eval_count = 1

官方 CLASS-ID 是 1-based, 0-based 标签 = CLASS-ID - 1；类名用 rsplit('_', 1) 取前缀

泄漏检查（三层 + 内容级）

文件名交集	0 / 0 / 0
MD5 交集（防改名）	0 / 0 / 0
孤儿图误入	0
划分内部重复图	0

python datasets/audit_leakage.py

官方数据事实基线（全部实测）

7390 images/ 下的 jpg	3680 trainval.txt 行数	3669 test.txt 行数	12 / 25 猫 / 狗 类别数	1-based 官方 CLASS-ID
---	--	--	---	---

官方不存在 train.txt / val.txt，只有 trainval.txt 与 test.txt；train/val 必须自行分层切分

40 epoch 迁移训练把 test Top-1 做到 92.34%

AdamW · cosine + 3ep warmup · Label Smoothing 0.1 · bf16 autocast · batch 64 · seed 42

测试集 (3669 张, 只评价一次)

92.34%

Top-1 Accuracy

99.26%

Top-5 Accuracy

92.22%

Macro-F1

验证集 (740 张) best 选模依据 val_macro_f1 = 0.9462 @ epoch 34

训练检查

✓ 骨干确实被训练	706 张量
✓ 其中骨干张量	699 个
✓ test 只评价一次	eval_count=1
✓ 权重加载无异常	unexpected=0

1000 类 → 37 类: 怎么做、差异在哪

正确做法

```
timm.create_model(..., num_classes=37,  
distillation=False)
```

保留分类头的 BN 统计量 (与类别数无关的迁移先验)
missing = 2 (仅换头后的 Linear weight / bias)

禁止做法

```
model.reset_classifier(37)
```

会丢掉 head.head.bn 的统计量; distillation=False 时
head_dist 属性被整体删除, 后续断言必然失败

四组优化各有假设，控制变量由工具强制校验

Baseline 有两个问题：样本仅 2940 张而参数 473 万（过拟合风险）；主干前期被大学习率推离预训练解

B 混合增强

3 个差异键

Mixup $\alpha=0.2$ + CutMix $\alpha=1.0$ ，每个 batch 都做

假设：混合两张图片与标签，尝试减少对训练样本的记忆

代价：需要更多 epoch 才能收敛

C 加强数据增强

8 个差异键

RandAugment($n=2, m=9$) + RRC scale 下限 $0.35 \rightarrow 0.25$

假设：覆盖旋转 / 剪切 / 色调变换，减少对局部纹理的依赖

风险：2940 张下容易过增强

D 差异化学习率

1 个差异键

主干倍率 $0.1 \rightarrow 0.05$ （有效 lr $1e-4 \rightarrow 5e-5$ ）

假设：新头快速收敛，让主干更新幅度更小

代价：训练曲线整体右移

A 组合式（基础任务所选项）

12 个差异键

B + C + D 全开

假设：三者机制互补，前段更稳 + 末段更高

风险：正则叠加会延长收敛

控制变量不是靠自觉，而是可执行校验

```
python tools/diff_config.py configs/baseline.yaml configs/opt_combo.yaml
```

叶子级比对，实际差异键必须与配置声明的 `_expected_diff_` 逐项一致

7 份配置全部 DIFF PASS (3 / 8 / 1 / 12 / 8 / 1 / 9 键)

```
python tools/same_budget.py → total_iters_equal=True (8 组均为 40 epoch × 45 step)
```

保持不变

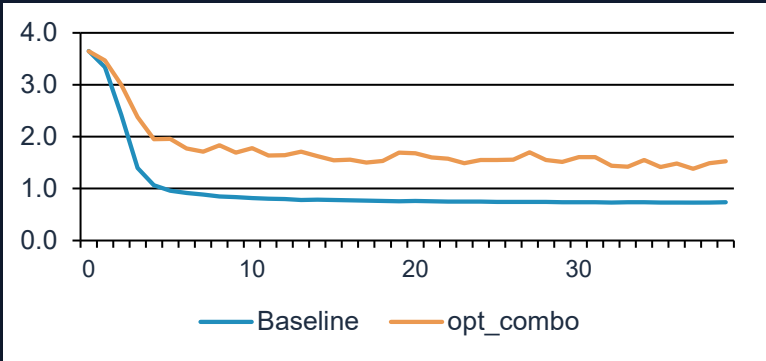
数据划分 · 预训练权重 · 种子

训练预算 · 评价方式 · 选模标准

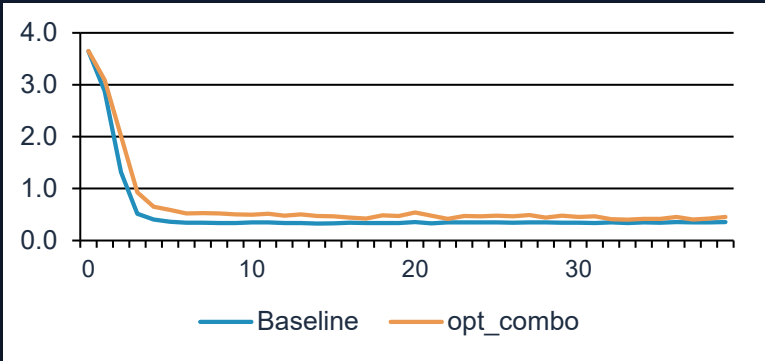
训练曲线：两组都收敛，组合方案未提高测试准确率

Pet-37 · Baseline vs opt_combo · 相同划分、seed=42、40 epoch · 每项指标共用坐标范围

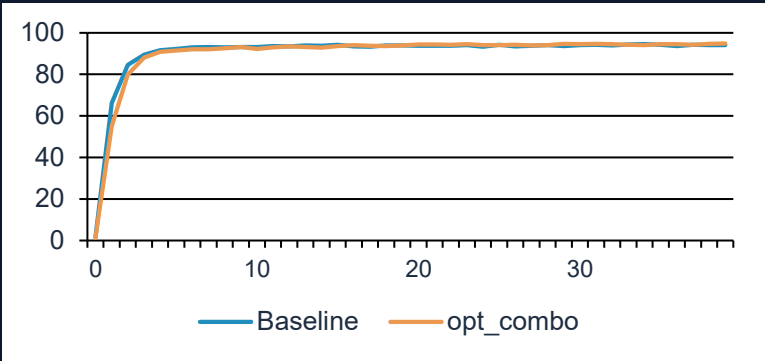
训练损失



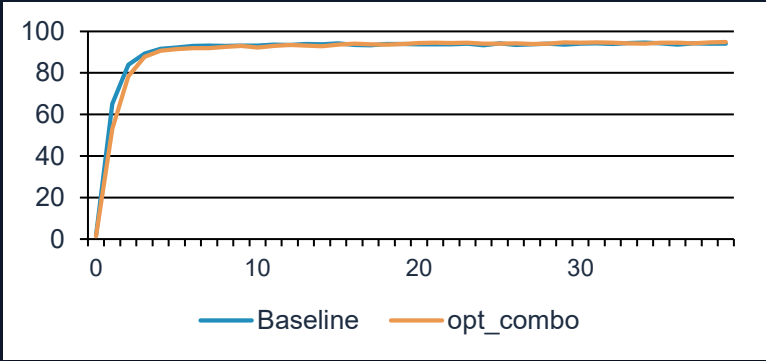
验证损失



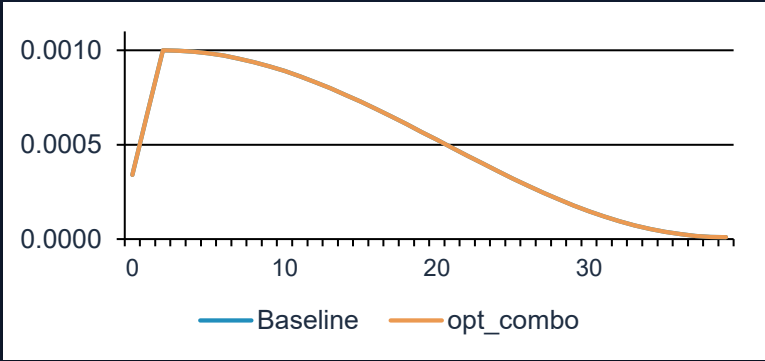
验证 Top-1 (%)



验证 Macro-F1 (%)



学习率



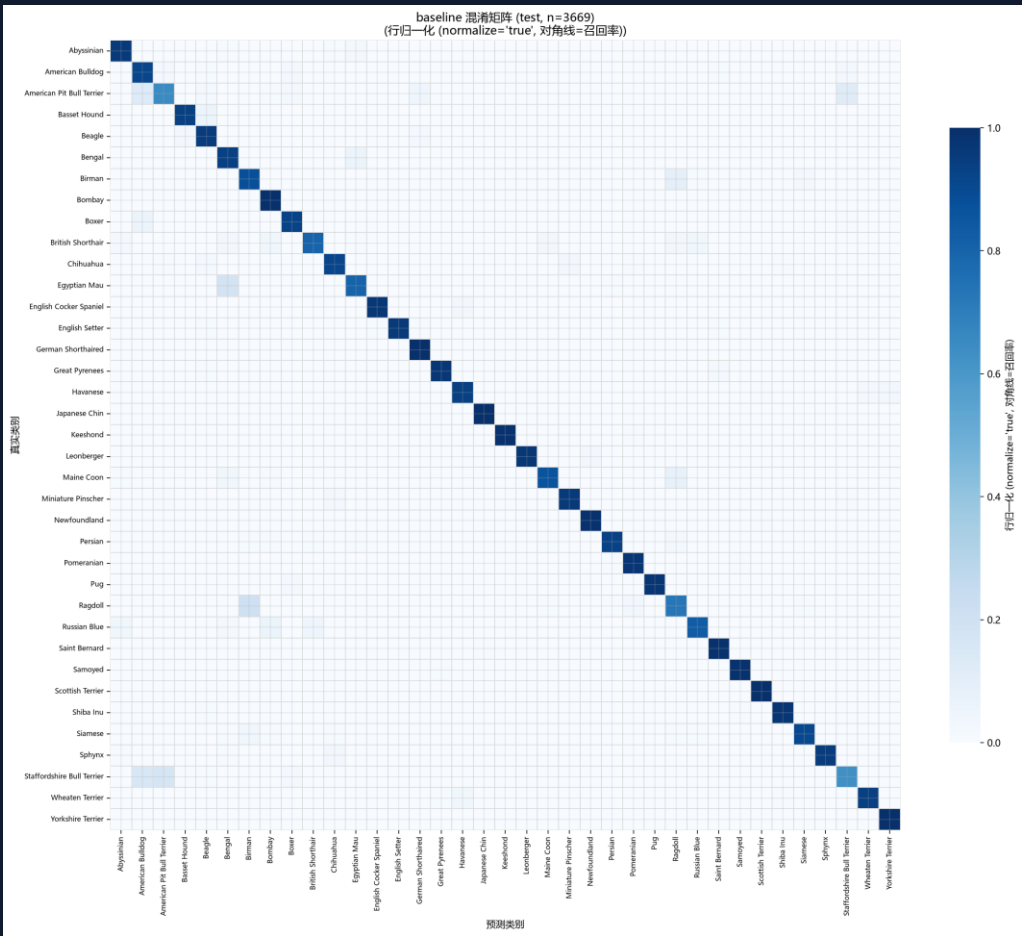
测试集结果 (3669 张)

指标	Baseline	组合
Top-1	92.34%	92.12%
Top-5	99.26%	99.59%
Macro-F1	92.22%	92.01%

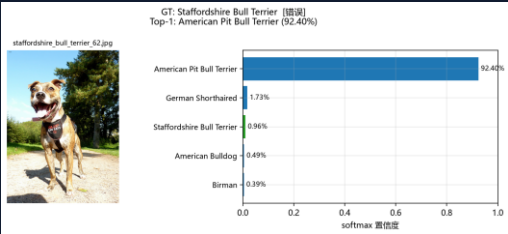
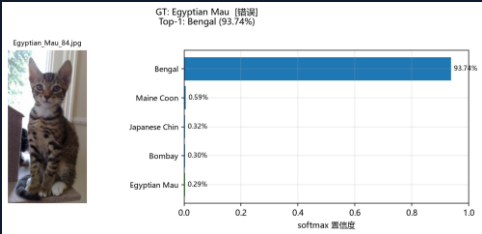
前几轮提升最快，后段逐渐稳定。
软标签改变 train loss，不能直接横比。
单种子结果不足以判断稳定提升。

错在哪里：多数是同物种内的品种混淆

Baseline · Pet-37 test · 3669 张 · 行归一化混淆矩阵；右侧保留失败案例



281 个错误中，10 个跨猫狗物种
其余 271 个是同物种品种判断错误



这些失败图提示：相似外形、遮挡和背景
都可能影响判断。不能仅凭热力图确定因果。

下一步：补充容易混淆的品种和不同拍摄角度。

Grad-CAM: 同时看判对和判错的图片

Baseline · 挂载 stages[-1].blocks[-1] · 原图 / 热力图 / 叠加图; 展示 2 个正确 + 2 个失败

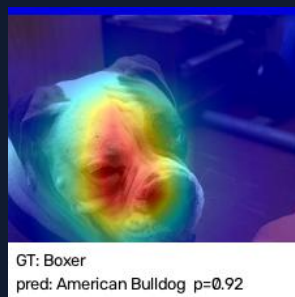
正确: Bengal



正确: Russian Blue



失败: Boxer



失败: Egyptian Mau

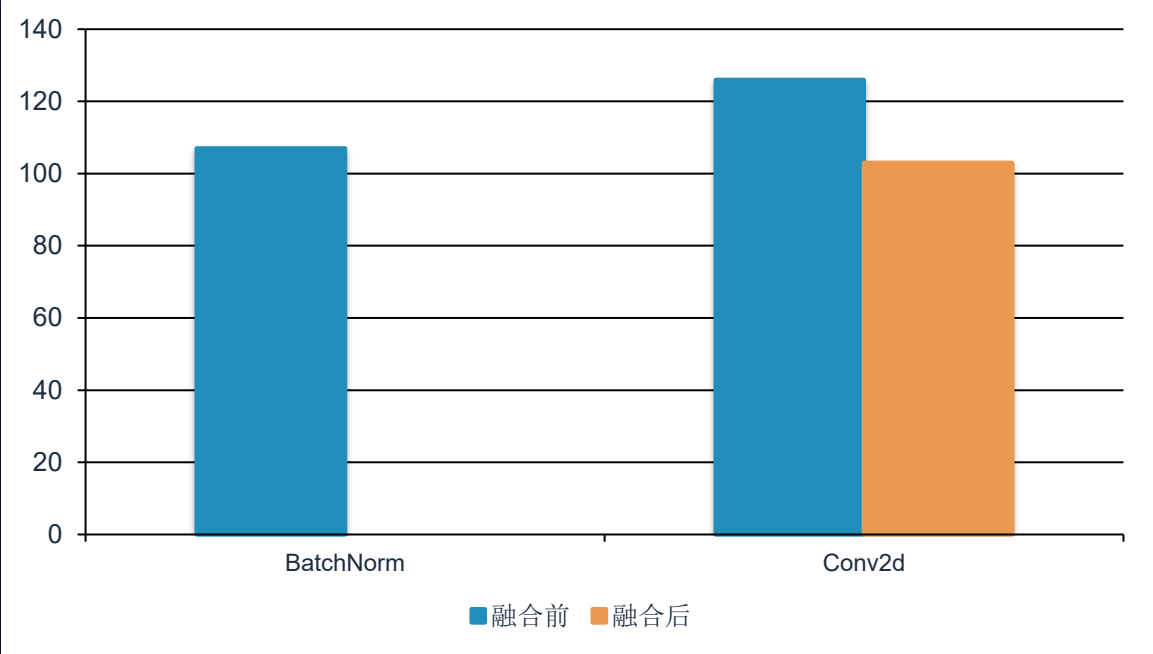


关注到主体 $\neq$ 品种一定判断正确。热力图用于检查线索, 仍要结合失败案例和完整测试集。

重参数化：合并分支后，误差仍在设定阈值内

Pet-37 / baseline_best.pt · CPU FP32 · eval → 深拷贝 → fuse · 固定随机输入 n=32, batch=8, 224×224, seed=20240912

PyTorch 模块数量



数值验证（随机输入）	结果
最大 $ \Delta \text{logits} $	7.093e-06
平均 $ \Delta \text{logits} $	2.031e-06
Top-1 一致	32 / 32
Top-5 最小交集	5 / 5
阈值 / 判定	max < 1e-4 / PASS

先把 BN 的固定统计量折进卷积，再把 1×1、3×3 与 identity 分支的核相加。

代数上等价；FP32 运算顺序改变会产生舍入误差。这里通过的是已测输入和设定阈值。

PyTorch ↔ ONNX: 固定 n=12, 比较同一输入张量

融合后的 PyTorch vs ORT CPUExecutionProvider · FP32 · batch=1 · 224×224 · 每个模型各 12 张

模型 / 输入列表	n	最大 $ \Delta\text{logits} $	平均 $ \Delta\text{logits} $	Top-1 / Top-5
M0.9 Pet-37 / pet_test	12	6.199e-06	1.501e-06	100% / 100%
M0.9 / ImageNet 子集	12	1.717e-05	2.360e-06	100% / 100%
M1.0 / ImageNet 子集	12	1.812e-05	2.464e-06	100% / 100%

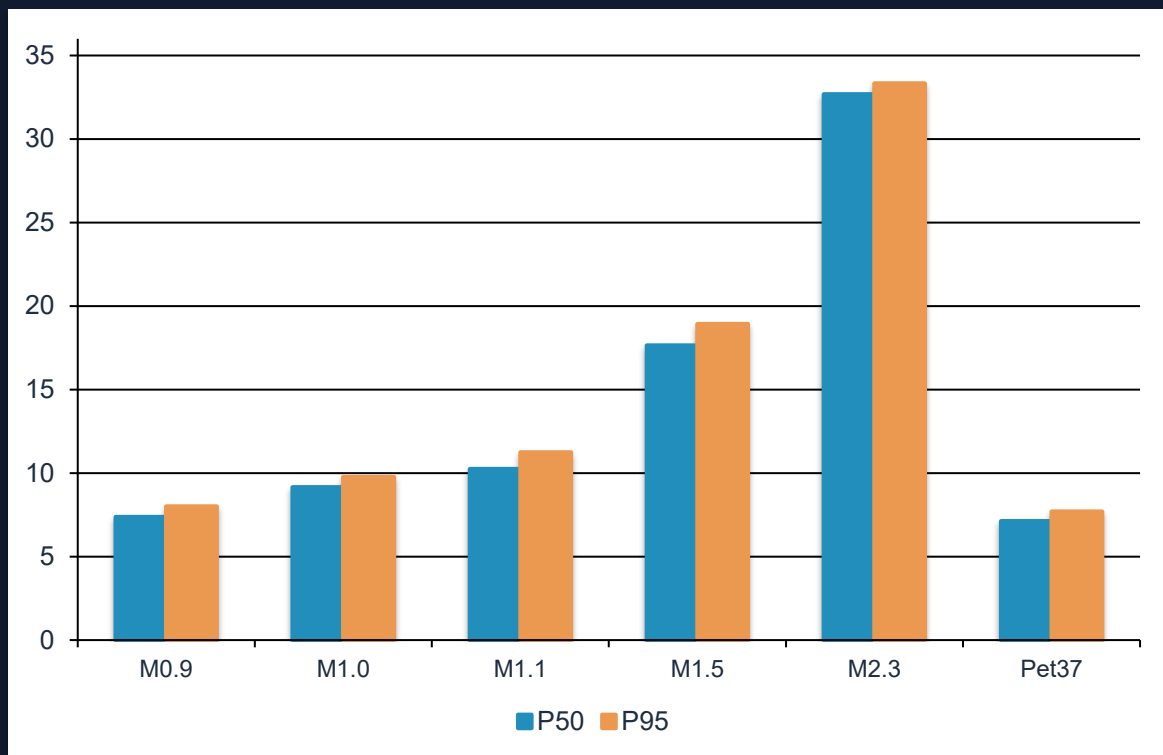
- 1 部署预处理由 PIL 实现；本次对比把同一张量送入两个推理后端。
- 2 结论仅覆盖这 12 张图片，不能写成完整测试集或全部六模型一致。
- 3 旧的 5.25e-06 缺少对应完整记录，当前统一引用上表落盘值。

复跑: `python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 --limit 12`
完整命令、固定列表与输出路径已写入本页备注和 report/LOGITS_AUDIT.md。

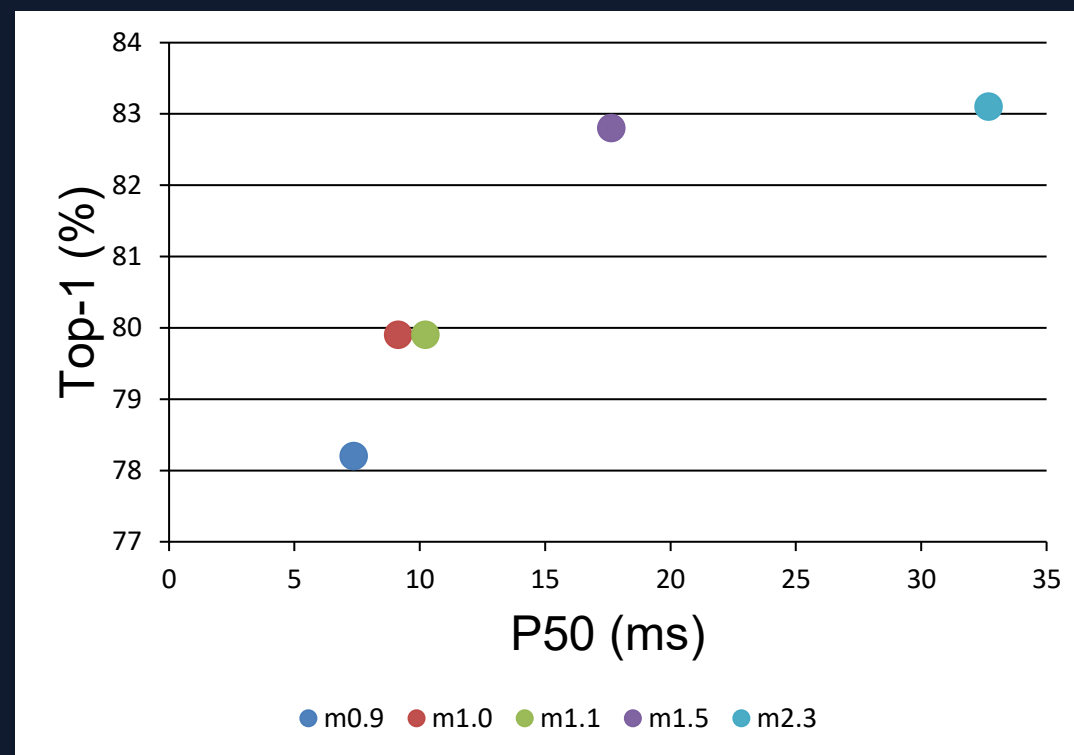
部署怎么选：先看同一机器上的延迟和准确率

i7-13650HX · Windows 11 · ORT 1.30.0 CPUExecutionProvider · FP32 · $1 \times 3 \times 224 \times 224$ · 预热10 + 正式50 · threads=4

六模型推理耗时 (ms)



同一 ImageNet 子集：速度与准确率



本子集上 M1.0 / M1.1 的 Top-1 相同，M1.0 更快。更大模型的额外耗时，需要结合使用场景判断。

复现链路完整、数字可溯源、控制变量有工程化校验

37 条验收项自检 0 失败；代码零写死绝对路径；关键数字与实验口径见 outputs/ 和审计说明

三个最有代表性的问题

- 01 官方 .pth 与 timm 键名交集为 0 (713 vs 389)
实测证明两条路线权重逐位等价 ($\max|\Delta\text{logits}| = 0$)，省去手写键转换器
- 02 data/provided/ 在 29 页试题 PDF 中从未定义
改判为结构性偏差，改用官方归档 + 自建子集，并在 README 与报告中显式声明

- 03 早停导致训练预算不等价 (opt_mix 25 epoch vs baseline 40 epoch, 差 37.5%)
处置：统一关闭早停，8 组实验全部 40 epoch 满预算；把预算等价性固化成可执行检查 same_budget.py

结论

- ① RepViT 确为纯卷积，M0.9 实测 78.20%，贴近官方 78.7%
- ② Pet-37 test Top-1 92.34%，跨物种错误仅 3.56%
- ③ 重参数化 32 个随机输入通过；ONNX 固定 $n=12$ 样本一致
- ④ 四项优化未显示稳定提升；需要多种子实验进一步判断

后续计划

INT8 量化 / 知识蒸馏 (M2.3 $\rightarrow$ M0.9)
集显后端 (本机无集显，需换设备)
置信度校准 (实测温度 $T = 0.615$)
七类鲁棒性扰动已跑，可扩展 severity 级